



Asl5

AZIENDA TUTELA DELLA SALUTE LIGURIA

SPECIFICHE TECNICHE

Sistema GatewayFSE

Validazione documenti tramite Gateway Regionale FSE

Tabella revisioni documento

Versione	Revisione	Data	Autore	Descrizione modifica
1.0	Prima stesura	20/02/2026	Francesco Matraxia	Prima redazione specifiche tecniche sistema GatewayFSE

Sommario

1. Introduzione	5
1.1 Scopo del documento.....	5
1.2 Campo di applicazione.....	5
1.3 Definizioni e acronimi.....	6
1.4 Riferimenti documentali.....	6
2. Panoramica del sistema.....	7
2.1 Descrizione generale	7
2.2 Contesto applicativo.....	7
2.3 Sistemi esterni coinvolti	7
Gateway Regionale FSE	7
Servizio token	8
Database Oracle	8
Filesystem applicativo	8
2.4 Stakeholder e attori.....	8
Attori tecnici	8
Attori applicativi	9
Utenti operativi indiretti.....	9
3. Architettura del sistema	10
3.1 Architettura logica	10
3.2 Architettura fisica	11
3.3 Componenti applicativi.....	12
GatewayFSE Web Application	12
Servlet DocumentsValidation	12
Repository Oracle	13
Utility HTTP	13
Utility TLS	13
Utility JWT.....	14
3.4 Tecnologie utilizzate	14
4. Componenti software.....	15
4.1 Servlet DocumentsValidation	15
Responsabilità principali.....	15
Parametri gestiti	15
4.2 Repository database.....	16

Responsabilità principali.....	16
Tabelle coinvolte	16
Monitoraggio richiesta	16
4.3 Utility HTTP	17
Funzioni principali.....	17
Access token	17
Validazione documento.....	17
4.4 Utility TLS.....	18
Responsabilità principali.....	18
Certificati	18
4.5 Generazione JWT.....	18
Token prodotti.....	18
Dati utilizzati	18
Selezione certificati	19
4.6 Gestione configurazione.....	19
4.7 Cifratura password DataSource.....	19
Meccanismo.....	20
5. Flussi applicativi.....	21
5.1 Ricezione richiesta di validazione	21
Flusso operativo	21
5.2 Acquisizione access token	21
Flusso operativo	21
5.3 Generazione JWT FSE	22
Input utilizzati	22
Flusso operativo	22
5.4 Configurazione TLS	22
Flusso operativo	22
5.5 Invocazione servizio Gateway Regionale.....	22
Dati inviati.....	23
Request body applicativo	23
Flusso operativo	23
5.6 Gestione risposta.....	23
Risposta positiva	23
Risposta errore Gateway	23
5.7 Logging e monitoraggio	24
Logging su file	24

Logging su database	24
5.8 Diagramma di sequenza	25
5.9 Diagramma di flusso	26
6. API REST	27
6.1 Endpoint applicativo.....	27
6.2 Parametri di input.....	27
Parametri	27
6.3 Struttura richiesta multipart.....	27
6.4 Struttura risposte.....	27
Risposta positiva	28
Risposta errore Gateway	28
Errore applicativo interno	28
6.5 Gestione errori	28
Gestione resilienza	28
7. Configurazione sistema	30
7.1 Configurazione Tomcat.....	30
Deploy applicazione.....	30
Avvio servizio	30
7.2 Configurazione DataSource JNDI	30
Parametri principali	31
7.3 Configurazione applicativa	31
Parametri principali	31
7.4 Configurazione certificati	31
Tipologie file	31
Selezione certificati per programma	32
7.5 Configurazione logging	32
Directory log	32
Modalità debug	32
7.6 Variabili d'ambiente	32
Configurazione Windows Server	32
Note di sicurezza.....	33
8. Database Oracle	34
8.1 Connessione	34
8.2 Tabelle di monitoraggio.....	34
MONITORAGGIO_GATEWAY	34
Stati principali.....	34

8.3 Tabelle di log.....	35
LOG_TABLE	35
8.4 Accessi richiesti.....	35
9. Sicurezza	36
9.1 Sicurezza trasporto	36
9.2 Gestione certificati	36
9.3 Gestione JWT	36
9.4 Cifratura password database.....	36
Algoritmo	37
Chiave	37
Formato consigliato nel context.....	37
9.5 Permessi filesystem	37
10. Performance e limiti	38
10.1 Dimensione file	38
10.2 Timeout chiamate esterne	38
10.3 Gestione concorrenza.....	38
10.4 Limiti operativi.....	39
11. Deployment	40
11.1 Build applicazione.....	40
11.2 Deploy su Tomcat 9	40
Procedura	40
11.3 Configurazione ambiente	40
12. Requisiti di sistema	42
12.1 Requisiti hardware.....	42
12.2 Requisiti software.....	42
12.3 Requisiti rete	42
13. Ambiente di esercizio	43
13.1 Componenti dell'ambiente.....	43
Server applicativo (PVESBASL5).....	43
Server database (TOPDBF).....	43
Servizi esterni (Liguria Digitale)	43
Filesystem applicativo (PVESBASL5)	43
13.2 Modalità di esecuzione.....	43
13.3 Considerazioni operative	44
13.4 Sintesi componenti.....	44

1. Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere in maniera dettagliata l'architettura, il funzionamento, le modalità di configurazione e le procedure di deployment del sistema **GatewayFSE**.

Il sistema GatewayFSE è un'applicazione Java deployata su Apache Tomcat 9, finalizzata all'integrazione con il servizio esterno del **Gateway Regionale FSE** per la validazione di documenti sanitari ricevuti in input.

In particolare, il documento fornisce le informazioni tecniche necessarie per:

- comprendere il funzionamento complessivo del sistema;
- descrivere i componenti applicativi coinvolti;
- documentare il flusso di validazione del documento;
- descrivere la generazione dei token JWT richiesti dal Gateway FSE;
- descrivere la configurazione TLS e l'utilizzo dei certificati;
- documentare la configurazione del DataSource Oracle su Tomcat;
- supportare attività di installazione, manutenzione, troubleshooting e gestione evolutiva.

Il documento è rivolto a personale tecnico, amministratori di sistema, sviluppatori e manutentori applicativi coinvolti nella gestione del sistema GatewayFSE.

1.2 Campo di applicazione

Il sistema GatewayFSE è utilizzato per validare documenti trasmessi verso il Gateway Regionale FSE.

La funzionalità principale consiste nella ricezione di una richiesta HTTP multipart contenente:

- codice fiscale del paziente;
- codice fiscale dell'operatore;
- tipo programma;
- file documentale da validare.

Il sistema utilizza tali informazioni per:

- registrare l'inizio della richiesta su database;
- ottenere un access token dal servizio di autenticazione configurato;
- generare i token JWT richiesti dal Gateway FSE;
- selezionare i certificati corretti in base al tipo programma;
- configurare il contesto TLS;
- invocare il servizio esterno di validazione;
- restituire al chiamante la risposta ricevuta;
- aggiornare il monitoraggio della richiesta su database.

Il sistema è deployato in ambiente **Windows Server 2022** tramite **Apache Tomcat 9** ed è sviluppato in **Java 21** con gestione build tramite **Maven**.

1.3 Definizioni e acronimi

Termine	Descrizione
FSE	Fascicolo Sanitario Elettronico
Gateway FSE	Servizio esterno regionale/nazionale utilizzato per la validazione dei documenti sanitari
JWT	JSON Web Token
TLS	Transport Layer Security
P12 / PKCS#12	Formato contenitore per certificati e chiavi private
PEM	Formato testuale per certificati o chiavi pubbliche
JNDI	Java Naming and Directory Interface
DataSource	Risorsa applicativa per la connessione al database
Tomcat	Application server utilizzato per il deploy dell'applicazione Java
WAR	Web Application Archive
Oracle DB	Database relazionale utilizzato dal sistema
Multipart	Formato HTTP utilizzato per inviare parametri e file nella stessa richiesta
CF	Codice fiscale
SSLContext	Contesto Java per la gestione delle connessioni TLS

1.4 Riferimenti documentali

Documento / Risorsa	Descrizione
Sorgenti applicazione GatewayFSE	Codice sorgente Java del progetto
LD23GPS-SA4053-001_NoteIntegrazione.docx	Note di integrazione fornite ai nodi
LD23ICT-PR6054-003_InterfacciaMiddlewareRegv1.2.0.docx	Specifiche di integrazione del Gateway regionale richiamato dell'applicativo
SwaggerMiddlewareFSE2Liguria.yaml	Yaml di riferimento del Middleware regionale

2. Panoramica del sistema

2.1 Descrizione generale

Il sistema GatewayFSE è un'applicazione web Java che espone una servlet dedicata alla validazione di documenti tramite il Gateway Regionale FSE.

L'applicazione riceve in input una richiesta multipart contenente il documento da validare e i metadati minimi necessari alla costruzione dei token di sicurezza richiesti dal Gateway. La servlet principale elabora la richiesta, ottiene un token di accesso, genera i JWT applicativi, configura il canale TLS e inoltra il documento al servizio esterno.

La servlet principale `DocumentsValidation` acquisisce dalla richiesta multipart i parametri `cf_paziente`, `cf_operatore`, `tipo_programma` e `file`, che rappresentano rispettivamente il codice fiscale del paziente, il codice fiscale dell'operatore, il programma applicativo di riferimento e il documento da validare.

Il sistema è progettato per funzionare come componente di integrazione server-side, senza interfaccia utente diretta. L'interazione avviene tramite chiamate HTTP verso l'endpoint applicativo esposto da Tomcat.

2.2 Contesto applicativo

Il sistema si inserisce in un contesto di integrazione con il Fascicolo Sanitario Elettronico e con il Gateway Regionale.

Il chiamante applicativo invia al sistema GatewayFSE un documento da validare e i dati necessari alla generazione dei token. GatewayFSE non effettua la validazione documentale internamente, ma svolge il ruolo di componente di intermediazione tecnica verso il servizio esterno.

Le responsabilità principali del sistema sono:

- normalizzare e verificare i parametri ricevuti;
- produrre le credenziali applicative richieste dal servizio esterno;
- presentare i certificati corretti;
- invocare il servizio di validazione;
- restituire una risposta JSON strutturata;
- mantenere tracciabilità dell'intero ciclo di richiesta.

2.3 Sistemi esterni coinvolti

Il sistema GatewayFSE interagisce con i seguenti sistemi esterni:

Gateway Regionale FSE

Servizio esterno invocato per la validazione del documento ricevuto in input.

Il sistema GatewayFSE invia al Gateway:

- access token;
- JWT di autorizzazione;
- JWT di firma;
- documento allegato;
- request body applicativo;
- certificato client tramite TLS.

Servizio token

Servizio esterno utilizzato per ottenere l'access token tramite grant type `client_credentials`.

La classe `HttpUtility` costruisce la richiesta POST con content type `application/x-www-form-urlencoded` e parametro `grant_type=client_credentials`; in caso di esito positivo deserializza la risposta in `TokenResponse`, mentre in caso di errore deserializza in `ErrorTokenResponse`.

Database Oracle

Database utilizzato per:

- logging applicativo;
- monitoraggio dell'inizio e della fine delle richieste;
- registrazione degli esiti;
- tracciamento dei dati restituiti dal Gateway.

La connessione avviene tramite `DataSource` JNDI configurato su Tomcat e recuperato dall'applicazione tramite lookup `java:/comp/env/jdbc/FSEGatewayDataSource`.

Filesystem applicativo

Utilizzato per:

- caricamento configurazioni;
- lettura certificati;
- gestione temporanea del file ricevuto;
- scrittura log applicativi.

2.4 Stakeholder e attori

Attori tecnici

- **Amministratori di sistema**
Responsabili della gestione del server Windows, del servizio Tomcat, delle variabili d'ambiente, del filesystem e dei certificati.

- **DBA / amministratori database**
Responsabili della configurazione dell'utente Oracle, dei permessi sulle tabelle applicative e della disponibilità del database.
- **Sviluppatori / manutentori applicativi**
Responsabili della manutenzione del codice, della build Maven, della generazione del WAR e dell'evoluzione applicativa.

Attori applicativi

- **Sistema chiamante interno**
Invoca il servizio GatewayFSE trasmettendo il documento e i dati necessari alla validazione.
- **Gateway Regionale FSE**
Riceve la richiesta di validazione e restituisce l'esito.

Utenti operativi indiretti

Non sono previsti utenti finali diretti. L'utilizzo del sistema avviene tramite integrazione applicativa.

Eventuali accessi manuali sono limitati ad attività di:

- test tecnico;
 - diagnostica;
 - consultazione log;
 - verifica monitoraggio su database.
-

3. Architettura del sistema

3.1 Architettura logica

Il sistema GatewayFSE è strutturato secondo un'architettura a componenti applicativi, in cui ciascun modulo è responsabile di una specifica funzione del flusso di validazione.

I principali componenti logici sono:

- **Servlet di validazione**
 - riceve la richiesta HTTP multipart;
 - acquisisce parametri e file;
 - coordina il flusso applicativo.
- **Modulo di autenticazione**
 - richiede l'access token al servizio configurato;
 - gestisce risposte di successo ed errore.
- **Modulo JWT**
 - genera i token JWT richiesti dal Gateway;
 - costruisce claim e header in funzione del tipo programma;
 - seleziona certificati e issuer appropriati.
- **Modulo TLS**
 - carica il keystore PKCS#12;
 - costruisce il contesto SSL/TLS;
 - consente la mutua autenticazione verso il servizio esterno.
- **Modulo HTTP**
 - costruisce la richiesta multipart verso il Gateway;
 - invia documento, request body e header richiesti;
 - deserializza la risposta.
- **Modulo database**
 - registra inizio elaborazione;
 - aggiorna l'esito finale;
 - gestisce logging su database.
- **Modulo sicurezza DataSource**
 - decifra la password database configurata nel context Tomcat;
 - utilizza una chiave AES letta da variabile d'ambiente.

L'interazione tra i componenti avviene internamente all'applicazione Java, mentre l'integrazione con sistemi esterni avviene tramite:

- HTTP/HTTPS;
- TLS con certificato client;
- DataSource Oracle via JNDI;
- filesystem locale per certificati, configurazioni e log.

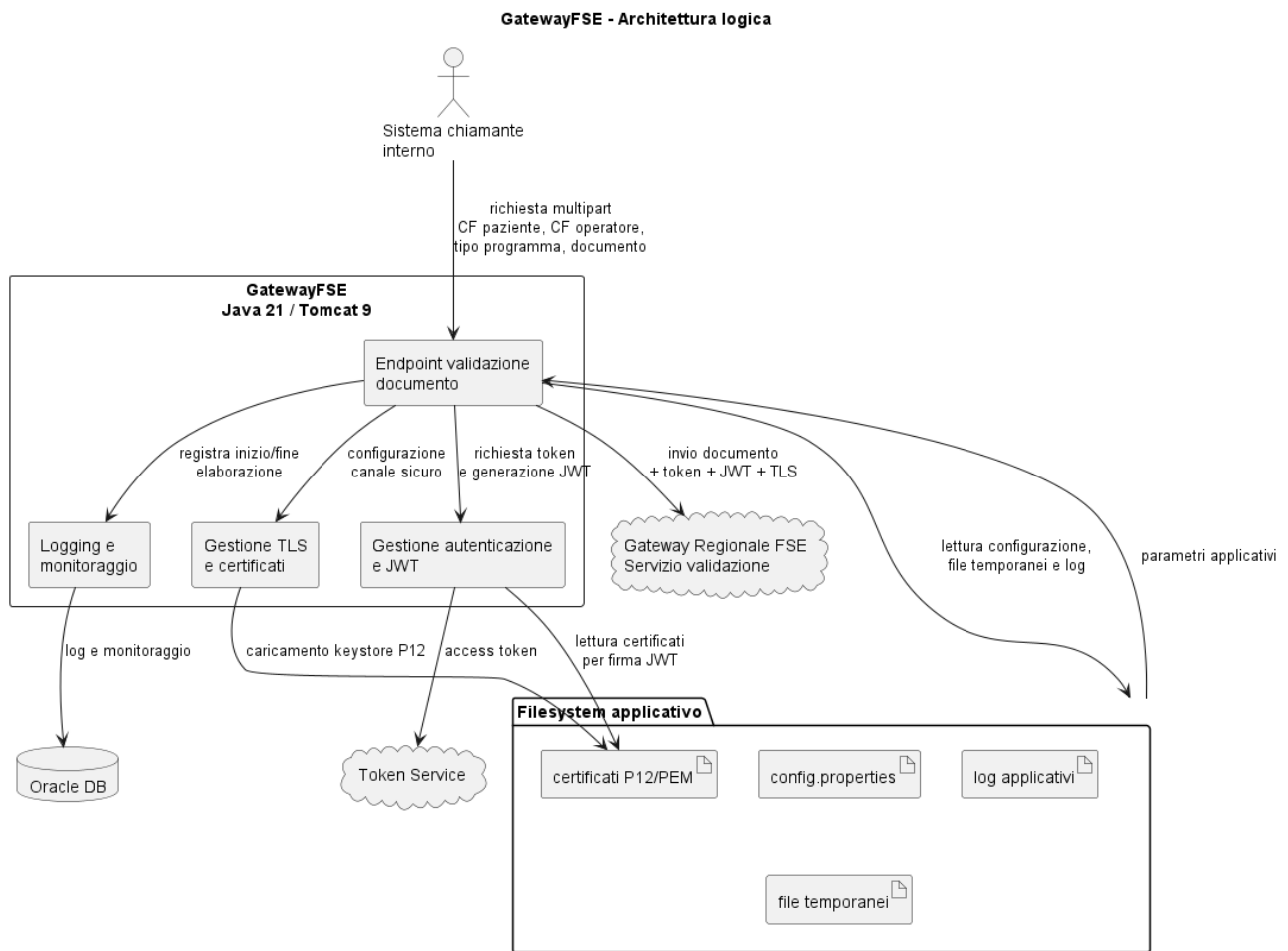


Immagine 1 – Architettura logica

3.2 Architettura fisica

Il sistema è installato su infrastruttura **Windows Server 2022** ed è deployato su **Apache Tomcat 9**.

Schema fisico di riferimento:

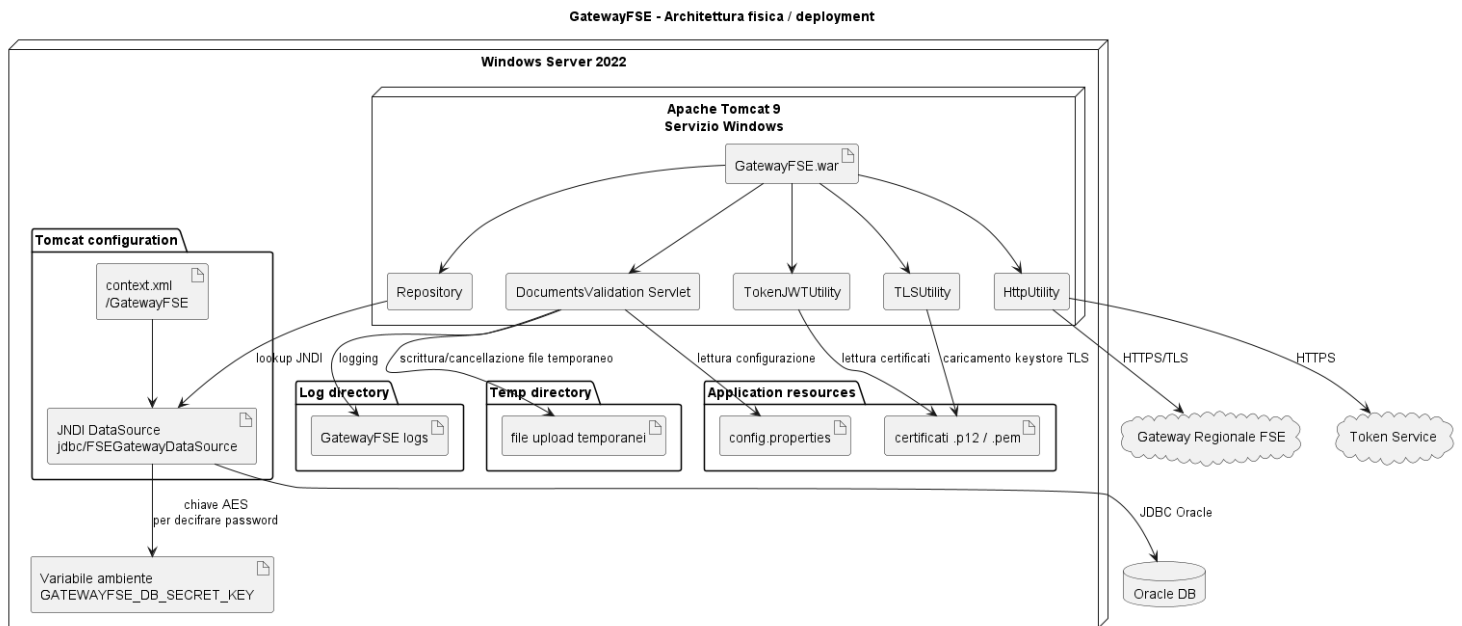


Immagine 2 – Architettura fisica

La risorsa database è configurata tramite context Tomcat come DataSource JNDI.

L'applicazione accede al database Oracle esclusivamente tramite il DataSource:

```
java:/comp/env/jdbc/FSEGatewayDataSource
```

3.3 Componenti applicativi

GatewayFSE Web Application

Applicazione web Java deployata come WAR su Tomcat 9.

Responsabilità principali:

- esposizione endpoint di validazione;
- ricezione file multipart;
- gestione parametri applicativi;
- generazione token JWT;
- invocazione Gateway Regionale;
- gestione risposte;
- monitoraggio e logging.

Servlet DocumentsValidation

Componente centrale dell'applicazione.

Responsabilità principali:

- caricare configurazione e logger in fase di inizializzazione;
- inizializzare repository e utility JWT;
- leggere parametri della richiesta;
- salvare temporaneamente il file ricevuto;
- invocare token service;
- generare JWT;
- preparare TLS;
- chiamare il servizio esterno;
- aggiornare il monitoraggio database;
- restituire JSON al chiamante.

Repository Oracle

Componente di persistenza.

Responsabilità principali:

- lookup del DataSource JNDI;
- scrittura log su tabella LOG_TABLE;
- inserimento record iniziale su MONITORAGGIO_GATEWAY;
- aggiornamento record finale con esito, trace id, workflow id e messaggi di errore.

Il repository registra l'inizio dell'elaborazione inserendo una riga in MONITORAGGIO_GATEWAY con esito iniziale IN_PROGRESS .

Utility HTTP

Componente di integrazione HTTP.

Responsabilità principali:

- richiesta access token;
- costruzione POST verso endpoint esterni;
- invio multipart verso Gateway;
- gestione header applicativi;
- deserializzazione JSON di successo o errore.

Utility TLS

Componente dedicato alla costruzione del contesto SSL.

Responsabilità principali:

- caricamento keystore PKCS#12;
- inizializzazione KeyManagerFactory;
- inizializzazione TrustManagerFactory;
- costruzione SSLContext;
- abilitazione connessione TLS per il client HTTP.

La classe TLSUtility contiene metodi per configurare il TLS tramite file P12 e password, inizializzando KeyManagerFactory, TrustManagerFactory e SSLContext .

Utility JWT

Componente dedicato alla generazione dei token.

Responsabilità principali:

- scelta dei certificati in base al tipo programma;
- selezione password P12;
- estrazione chiave privata;
- costruzione JWT di autorizzazione;
- costruzione JWT di firma;
- valorizzazione claim FSE.

Il tipo programma viene utilizzato per selezionare configurazioni, certificati e issuer; ad esempio LDO, RSA, CERT_VACC, SING_VACC e altri casi vengono gestiti tramite switch applicativo .

3.4 Tecnologie utilizzate

Componente	Tecnologia	Ruolo
Sistema operativo	Windows Server 2022	Ambiente di esecuzione
Application Server	Apache Tomcat 9	Hosting applicazione Java
Linguaggio	Java 21	Implementazione applicativa
Build	Maven	Compilazione e packaging
Packaging	WAR	Deploy applicazione web
Database	Oracle	Persistenza log e monitoraggio
Connessione DB	JNDI DataSource	Accesso database gestito da Tomcat
Sicurezza token	JWT RS256	Autorizzazione e firma verso Gateway
TLS	SSLContext Java / PKCS#12	Connessione sicura con certificato client
HTTP Client	OkHttp / HttpURLConnection	Invocazione servizi esterni
JSON	Jackson ObjectMapper	Serializzazione/deserializzazione
Logging	java.util.logging + DB log	Log applicativi

4. Componenti software

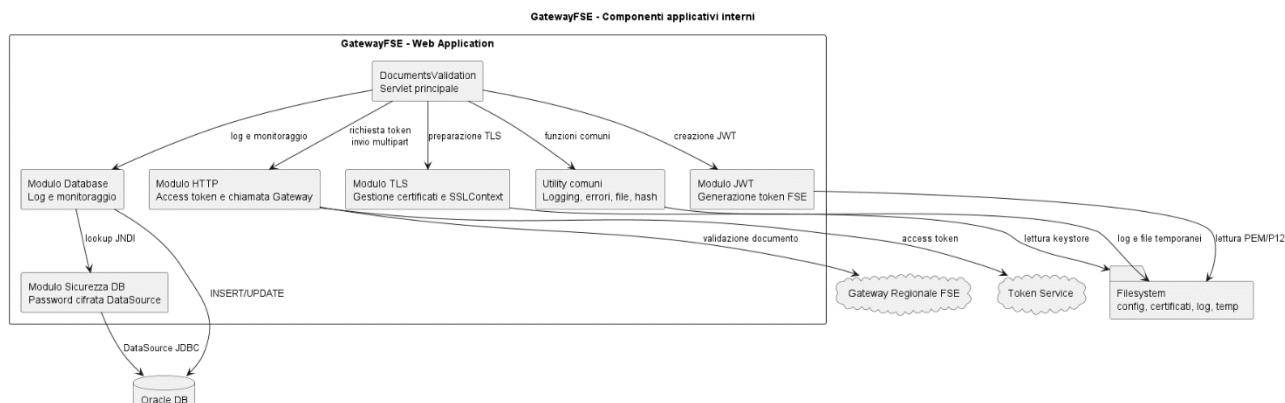


Immagine 3 – Diagramma dei componenti

Il diagramma rappresenta la suddivisione interna dell'applicazione GatewayFSE nei principali moduli funzionali. La servlet `DocumentsValidation` coordina il flusso applicativo, demandando ai moduli specializzati la gestione delle chiamate HTTP, la generazione dei JWT, la configurazione TLS, l'accesso al database, la gestione della password cifrata del DataSource e le funzioni comuni di logging, gestione errori e accesso ai file.

4.1 Servlet DocumentsValidation

La servlet `DocumentsValidation` rappresenta il punto di ingresso applicativo del sistema.

La servlet è configurata per accettare richieste multipart tramite annotazione `@MultipartConfig`, necessaria per ricevere file e parametri nella stessa richiesta HTTP.

Responsabilità principali

- gestione richieste HTTP GET e POST;
- lettura parametri multipart;
- validazione input;
- salvataggio temporaneo file;
- richiesta access token;
- generazione JWT;
- configurazione TLS;
- invocazione Gateway;
- restituzione risposta JSON;
- cancellazione file temporaneo;
- aggiornamento monitoraggio.

Parametri gestiti

Parametro	Descrizione	Obbligatorio
cf_paziente	Codice fiscale del paziente	Sì
cf_operatore	Codice fiscale dell'operatore	Sì

tipo_programma	Tipo programma/documento	Sì
file	Documento da validare	Sì

La servlet verifica la presenza dei parametri richiesti e, in caso di dati mancanti o invalidi, restituisce errore HTTP 400 tramite `Utility.handleBadRequest`.

4.2 Repository database

La classe `Repository` gestisce l'accesso al database Oracle.

Il `DataSource` non viene creato direttamente dal codice applicativo, ma recuperato tramite JNDI:

```
Context ctx = new InitialContext();
dataSource = (DataSource)
ctx.lookup("java:/comp/env/jdbc/FSEGatewayDataSource");
```

Responsabilità principali

- recupero `DataSource` JNDI;
- inserimento log applicativi;
- registrazione inizio elaborazione;
- aggiornamento fine elaborazione;
- fornitura connessioni database.

Tabelle coinvolte

Tabella	Utilizzo
LOG_TABLE	Registrazione log applicativi
MONITORAGGIO_GATEWAY	Tracciamento richieste verso Gateway

Monitoraggio richiesta

All'inizio della chiamata viene inserito un record con:

- data e ora;
- servizio chiamante;
- esito iniziale `IN_PROGRESS`;
- codice fiscale paziente;
- codice fiscale operatore;
- programma.

A fine elaborazione lo stesso record viene aggiornato con:

- esito finale;
- status Gateway;
- title;
- detail;
- trace id;

- workflow instance id;
- messaggio errore;
- eventuale JSON completo.

La classe `Repository` aggiorna la tabella `MONITORAGGIO_GATEWAY` valorizzando campi come `ESITO`, `GATEWAY_STATUS`, `GATEWAY_TITLE`, `GATEWAY_DETAIL`, `GATEWAY_TRACE_ID` e `GATEWAY_WORKFLOW_INSTANCE_ID`.

4.3 Utility HTTP

La classe `HttpUtility` gestisce le comunicazioni HTTP verso i servizi esterni.

Funzioni principali

- ottenere access token;
- inviare richieste POST;
- costruire multipart request;
- aggiungere header FSE;
- gestire client HTTPS;
- deserializzare risposte JSON.

Access token

Il metodo `getAccessToken`:

1. costruisce una richiesta POST;
2. imposta `grant_type=client_credentials`;
3. invia Authorization header;
4. in caso di HTTP 200 restituisce `TokenResponse`;
5. in caso di errore restituisce `ErrorTokenResponse`.

Validazione documento

Il metodo `postMultipartRequest` invia al Gateway:

- request body;
- file allegato;
- access token;
- `FSE-JWT-Authorization`;
- `FSE-JWT-Signature`;
- contesto SSL.

La richiesta multipart verso il Gateway viene costruita aggiungendo la parte `requestBody` e la parte `file`, con header `Authorization`, `FSE-JWT-Authorization`, `FSE-JWT-Signature` e `Accept: application/json`.

4.4 Utility TLS

La classe `TLSUtility` gestisce la configurazione TLS necessaria alla comunicazione sicura con il Gateway.

Responsabilità principali

- caricamento file P12;
- caricamento password keystore;
- inizializzazione chiavi client;
- creazione `SSLContext`;
- supporto a connessioni HTTPS con certificato client.

Certificati

Il sistema utilizza certificati differenti in base al tipo programma. I path dei certificati vengono risolti partendo dalle proprietà configurate e dal contesto servlet.

La gestione dei certificati è integrata con la generazione JWT, poiché i certificati sono utilizzati sia per presentazione TLS sia per firma dei token.

4.5 Generazione JWT

La classe `TokenJWTUtility` è responsabile della creazione dei token JWT richiesti dal Gateway.

Token prodotti

Il sistema produce due token principali:

Token	Utilizzo
JWT Authorization	Token di autorizzazione applicativa
FSE-JWT-Signature	Token di firma/integrità

Dati utilizzati

La generazione dei JWT utilizza:

- codice fiscale paziente;
- codice fiscale operatore;
- tipo programma;
- issuer;
- audience;
- subject role;
- purpose of use;
- patient consent;
- resource HL7 type;
- certificato pubblico;
- chiave privata contenuta nel P12.

Selezione certificati

Il parametro `tipo_programma` consente di determinare:

- issuer da utilizzare;
- subject application id;
- subject application vendor;
- subject application version;
- certificato PEM;
- certificato P12;
- password P12.

La classe `TokenMap` definisce i tipi programma supportati, tra cui `LDO`, `CERT_VACC_SKIPPER`, `SING_VACC_SKIPPER`, `SING_VACC_PHTRACK`, `CERT_VACC`, `SING_VACC` e `RSA`.

4.6 Gestione configurazione

La configurazione applicativa viene caricata da file:

`config.properties`

Il file viene letto dalla servlet in fase di inizializzazione tramite classloader.

Le proprietà configurano, tra le altre cose:

- URL servizio token;
- Authorization header;
- URL servizio validazione;
- path certificati;
- password keystore;
- parametri JWT;
- path log;
- modalità debug.

La servlet inizializza `config.properties` in `init()` e, dopo il caricamento, configura logger, repository e utility JWT.

4.7 Cifratura password DataSource

La password del database non viene mantenuta in chiaro nel context Tomcat.

Il sistema utilizza:

- `EncryptedDataSourceFactory`;
- `PasswordCrypto`;
- chiave AES-256 da variabile d'ambiente;

- password cifrata nel file context.

Meccanismo

1. Nel context Tomcat viene inserita la password cifrata.
2. Tomcat usa una factory custom per creare il DataSource.
3. La factory legge il valore della password.
4. La password viene decifrata tramite `PasswordCrypto`.
5. Il valore decifrato viene passato al DataSource Tomcat.
6. L'applicazione continua a usare il normale lookup JNDI.

La factory custom estende `org.apache.tomcat.jdbc.pool.DataSourceFactory`, legge l'attributo `password`, lo decifra tramite `PasswordCrypto.decrypt` e sostituisce il valore nella reference prima di delegare alla factory Tomcat standard.

La classe `PasswordCrypto` utilizza AES/GCM/NoPadding con IV di 12 byte e tag GCM a 128 bit; la chiave viene letta dalla variabile d'ambiente `GATEWAYFSE_DB_SECRET_KEY`.

5. Flussi applicativi

5.1 Ricezione richiesta di validazione

Il flusso inizia con una richiesta HTTP multipart verso la servlet applicativa.

La richiesta deve contenere:

- `cf_paziente;`
- `cf_operatore;`
- `tipo_programma;`
- `file.`

Flusso operativo

1. Il sistema riceve la richiesta.
2. Viene recuperato il path della servlet.
3. Viene avviato il logging applicativo.
4. Vengono letti i parametri multipart.
5. Viene inserito un record iniziale in `MONITORAGGIO_GATEWAY`.
6. Viene acquisito il file PDF.
7. Il file viene salvato temporaneamente nella directory temporanea Java.
8. Vengono verificati i parametri obbligatori.

Il file ricevuto viene salvato temporaneamente usando `System.getProperty("java.io.tmpdir")` e successivamente copiato tramite `Files.copy`.

5.2 Acquisizione access token

Dopo la validazione dell'input, il sistema richiede un access token al servizio configurato.

Flusso operativo

1. Recupero `token.url` da configurazione.
2. Recupero `authorization.header` da configurazione.
3. Invocazione `HttpUtility.getAccessToken`.
4. Gestione risposta:
 - successo: prosegue il flusso;
 - errore: restituisce HTTP 401 e aggiorna monitoraggio;
 - eccezione: restituisce HTTP 500 e aggiorna monitoraggio.

In caso di errore di autenticazione, il sistema restituisce la risposta `ErrorTokenResponse`, imposta lo stato HTTP 401 e aggiorna il monitoraggio con esito `ERROR`.

5.3 Generazione JWT FSE

Una volta ottenuto l'access token, il sistema genera i JWT necessari per invocare il Gateway FSE.

Input utilizzati

- tipo programma;
- codice fiscale paziente;
- codice fiscale operatore;
- proprietà applicative;
- servlet context.

Flusso operativo

1. Invocazione `TokenJWTUtility.CreaToken`.
2. Selezione configurazione in base a `tipo_programma`.
3. Recupero path certificati PEM/P12.
4. Caricamento certificati.
5. Estrazione chiave privata.
6. Generazione JWT authorization.
7. Generazione JWT signature.
8. Restituzione `TokenResponseDTO`.

La classe `TokenResponseDTO` incapsula i due valori generati: `authorizationBearer` e `fseJwtSignature`.

5.4 Configurazione TLS

Il sistema configura il contesto TLS prima dell'invocazione del Gateway.

Flusso operativo

1. Recupera il path relativo del file P12 da configurazione.
2. Risolve il path assoluto tramite `ServletContext.getRealPath`.
3. Carica il keystore PKCS#12.
4. Recupera la password del keystore da configurazione.
5. Costruisce `SSLContext`.
6. Passa il contesto SSL al client HTTP.

La servlet recupera `path.FileP12`, risolve il path assoluto e invoca `TLSUtility.prepareSSLContext` prima di chiamare l'endpoint di validazione.

5.5 Invocazione servizio Gateway Regionale

Dopo aver ottenuto access token, JWT e contesto TLS, il sistema invoca l'endpoint esterno di validazione.

Dati inviati

Dato	Descrizione
Access token	Token OAuth2 ottenuto dal servizio token
FSE-JWT-Authorization	JWT di autorizzazione
FSE-JWT-Signature	JWT di firma
File	Documento ricevuto in input
Request body	JSON statico per modalità validation
SSLContext	Contesto TLS con certificato client

Request body applicativo

Il sistema costruisce un JSON di richiesta:

```
{
  "healthDataFormat": "CDA",
  "mode": "ATTACHMENT",
  "activity": "VALIDATION"
}
```

Flusso operativo

1. Viene costruita una richiesta multipart.
2. Viene allegato il documento.
3. Vengono aggiunti header Authorization e FSE.
4. Viene inviata la richiesta HTTPS.
5. La risposta viene deserializzata:
 - o `ValidationResDTO` in caso di successo;
 - o `ValidationErrorResponseDTO` in caso di errore.

5.6 Gestione risposta

Il sistema gestisce due tipologie principali di risposta dal Gateway:

Risposta positiva

In caso di risposta positiva:

- viene impostato HTTP 200;
- viene loggata la risposta;
- viene aggiornato `MONITORAGGIO_GATEWAY` con esito `SUCCESS`;
- vengono salvati `warning`, `traceID` e `workflowInstanceId`;
- viene restituito JSON al chiamante.

La risposta positiva è rappresentata da `ValidationResDTO`, che contiene `traceID`, `spanID`, `workflowInstanceId` e `warning`.

Risposta errore Gateway

In caso di risposta di errore:

- viene impostato HTTP 500;
- viene loggata la risposta;
- viene aggiornato `MONITORAGGIO_GATEWAY` con esito `ERROR`;
- vengono salvati status, title, detail, traceID e workflowInstanceId;
- viene restituito JSON di errore al chiamante.

La risposta di errore è rappresentata da `ValidationErrorResponseDTO`, che contiene campi quali `traceID`, `spanID`, `type`, `title`, `detail`, `status`, `instance`, `workflowInstanceId` e `govway_id`.

5.7 Logging e monitoraggio

Il sistema utilizza due livelli di logging:

Logging su file

Gestito tramite `java.util.logging`, configurato da `Utility.initializeLogger`.

Caratteristiche:

- file giornaliero;
- directory configurabile;
- livello configurabile tramite `debug.mode`;
- rotazione per dimensione;
- log separato per classe.

La utility di logging crea la directory di log, genera un file basato sulla data e sulla classe, e configura `FileHandler` con rotazione a 10 MB e 5 file .

Logging su database

Gestito tramite `Repository.LogDB`.

Registra:

- severità;
 - messaggio;
 - funzione;
 - timestamp;
 - parametri;
 - servizio;
 - dati aggiuntivi.
-

5.8 Diagramma di sequenza

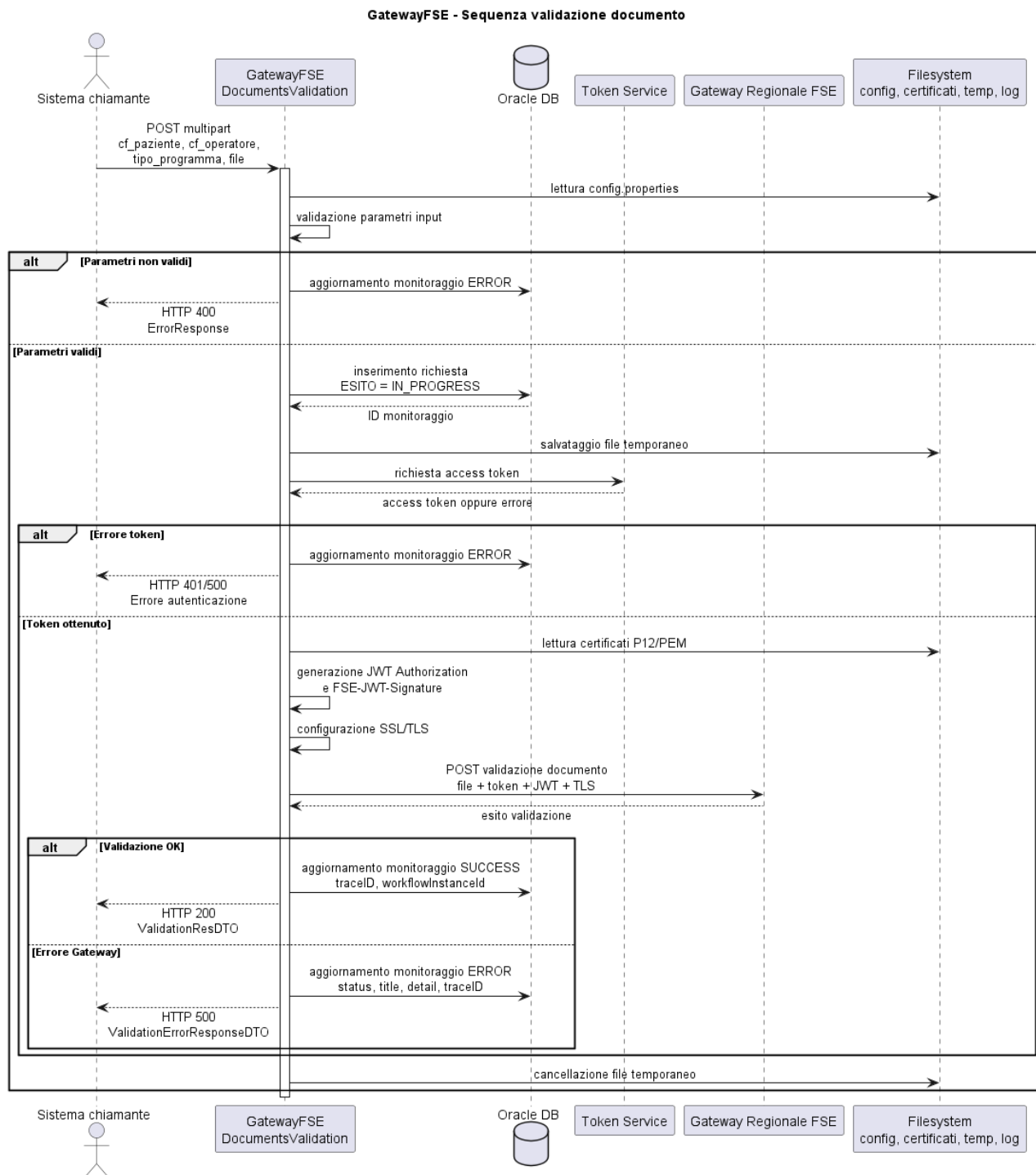


Immagine 4 – Diagramma di sequenza

5.9 Diagramma di flusso

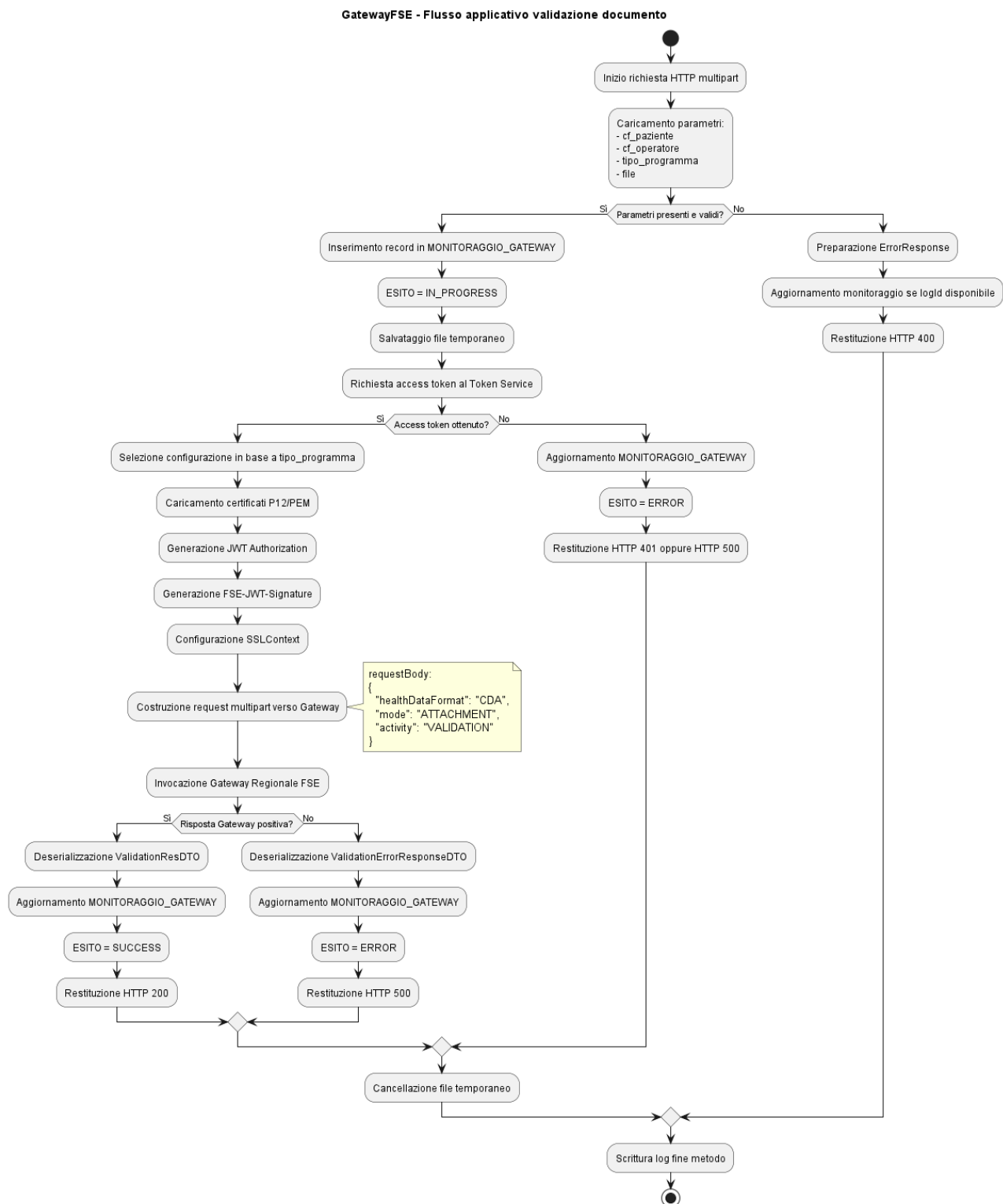


Immagine 5 – Diagramma di flusso

6. API REST

6.1 Endpoint applicativo

L'applicazione espone un endpoint servlet utilizzabile tramite richieste HTTP GET/POST.

Metodo	Endpoint	Descrizione
POST	/GatewayFSE/[mapping servlet]	Validazione documento tramite Gateway FSE
GET	/GatewayFSE/[mapping servlet]	Stesso processo applicativo, se previsto dal mapping servlet

6.2 Parametri di input

La richiesta deve essere di tipo:

multipart/form-data

Parametri

Nome	Tipo	Descrizione
cf_paziente	String	Codice fiscale del paziente
cf_operatore	String	Codice fiscale dell'operatore
tipo_programma	String	Tipologia programma/documento
file	File	Documento da validare

6.3 Struttura richiesta multipart

Esempio concettuale:

```
POST /GatewayFSE/DocumentsValidation HTTP/1.1
Content-Type: multipart/form-data
```

```
cf_paziente=<codice fiscale paziente>
cf_operatore=<codice fiscale operatore>
tipo_programma=<tipo programma>
file=<documento PDF/CDA>
```

Esempio tramite curl:

```
curl -X POST "http://SERVER:PORT/GatewayFSE/DocumentsValidation" \
-F "cf_paziente=RSSMRA80A01H501U" \
-F "cf_operatore=BNCLGU80A01H501X" \
-F "tipo_programma=CERT_VACC" \
-F "file=@documento.pdf"
```

6.4 Struttura risposte

Risposta positiva

Esempio:

```
{
  "traceID": "string",
  "spanID": "string",
  "workflowInstanceId": "string",
  "warning": "string"
}
```

Risposta errore Gateway

Esempio:

```
{
  "traceID": "string",
  "spanID": "string",
  "type": "string",
  "title": "string",
  "detail": "string",
  "status": 500,
  "instance": "string",
  "workflowInstanceId": "string",
  "govway_id": "string"
}
```

Errore applicativo interno

Esempio:

```
{
  "errorCode": 500,
  "errorMessage": "Descrizione errore"
}
```

La classe `ErrorResponse` restituisce un codice errore e un messaggio tramite i campi `errorCode` ed `errorMessage`.

6.5 Gestione errori

Codice HTTP	Significato	Condizione
200	Operazione completata	Validazione eseguita con successo
400	Bad Request	Parametri mancanti o invalidi
401	Unauthorized	Errore in fase di ottenimento token
500	Internal Server Error	Errore Gateway o errore applicativo interno

Gestione resilienza

Il sistema:

- registra sempre l'avvio della chiamata quando possibile;
 - aggiorna il monitoraggio anche in caso di errore;
 - restituisce JSON strutturato;
 - elimina il file temporaneo nel blocco `finally`;
 - scrive log applicativi e, quando disponibile, log su database.
-

7. Configurazione sistema

7.1 Configurazione Tomcat

L'applicazione è deployata su Apache Tomcat 9.

Deploy applicazione

Elemento	Valore
Tipo pacchetto	WAR
Directory deploy	<TOMCAT_HOME>\webapps\
Context path	/GatewayFSE
Runtime Java	Java 21
Sistema operativo	Windows Server 2022

Avvio servizio

Tomcat è configurato come servizio Windows.

Il servizio deve essere riavviato dopo modifiche a:

- context Tomcat;
- variabili d'ambiente;
- certificati;
- file WAR;
- configurazioni globali.

7.2 Configurazione DataSource JNDI

Il database Oracle è configurato tramite risorsa JNDI nel context Tomcat.

Esempio:

```
<Context path="/GatewayFSE">

  <Resource name="jdbc/FSEGatewayDataSource"
    auth="Container"
    type="javax.sql.DataSource"
    factory="com.mycompany.gatewayfse.EncryptedDataSourceFactory"
    driverClassName="oracle.jdbc.OracleDriver"
    url="jdbc:oracle:thin:@//HOST_DB:PORT/SERVICE_NAME"
    username="DB_USERNAME"
    password="ENC (PASSWORD_CIFRATA) "
    maxActive="10"
    initialSize="5"
    validationQuery="SELECT 1 FROM DUAL"
    testOnBorrow="true"
    useJmx="false"/>
```


</Context>

Parametri principali

Parametro	Descrizione
name	Nome JNDI della risorsa
factory	Factory custom per decifratura password
driverClassName	Driver JDBC Oracle
url	Stringa connessione Oracle
username	Utente database
password	Password cifrata
validationQuery	Query di validazione connessione
testOnBorrow	Verifica connessione prima dell'uso
maxActive	Numero massimo connessioni attive
initialSize	Connessioni iniziali del pool

7.3 Configurazione applicativa

Il file principale di configurazione è:

config.properties

Parametri principali

Parametro	Descrizione
token.url	URL servizio token
authorization.header	Header Authorization per richiesta token
url.validation	URL endpoint Gateway di validazione
path.FileP12	Path relativo file P12
ssl.keystore.password	Password keystore TLS
log.path	Directory log applicativi
debug.mode	Abilitazione livello debug
pem.*	Path file PEM per tipo programma
p12.*	Path file P12 per tipo programma
p12.*.password	Password certificati per tipo programma

7.4 Configurazione certificati

I certificati sono utilizzati per:

- autenticazione TLS;
- firma JWT;
- costruzione header x5c;
- selezione identità applicativa in base al tipo programma.

Tipologie file

Tipo file	Utilizzo
.p12	Contiene chiave privata e certificato
.pem	Contiene certificato pubblico
.properties	Definisce path e password

Selezione certificati per programma

Il parametro `tipo_programma` determina quale coppia PEM/P12 utilizzare.

Esempi di tipi programma:

- LDO;
- RSA;
- CERT_VACC;
- SING_VACC;
- CERT_VACC_SKIPPER;
- SING_VACC_SKIPPER;
- SING_VACC_PHTRACK.

7.5 Configurazione logging

Il logging è configurato tramite proprietà applicative.

Directory log

```
log.path=<directory_log>
```

Modalità debug

```
debug.mode=true|false
```

Se `debug.mode=true`, il livello log viene impostato a `FINE`.

Se `debug.mode=false`, il livello log viene impostato a `INFO`.

7.6 Variabili d'ambiente

Il sistema utilizza una variabile d'ambiente per la chiave di decifratura della password database:

```
GATEWAYFSE_DB_SECRET_KEY
```

Configurazione Windows Server

Esempio:

```
setx GATEWAYFSE_DB_SECRET_KEY "CHIAVE_AES_BASE64" /M
```

Dopo la configurazione è necessario riavviare il servizio Tomcat.

Note di sicurezza

La chiave:

- non deve essere committata nel repository;
 - non deve essere inserita nel WAR;
 - non deve essere scritta nei log;
 - deve essere accessibile solo agli amministratori autorizzati;
 - deve essere protetta tramite policy di sistema e permessi server.
-

8. Database Oracle

8.1 Connessione

La connessione Oracle avviene tramite DataSource JNDI gestito dal container Tomcat.

L'applicazione non contiene direttamente:

- host database;
- username reale;
- password in chiaro.

Questi parametri sono gestiti dal context Tomcat.

8.2 Tabelle di monitoraggio

MONITORAGGIO_GATEWAY

Tabella utilizzata per tracciare l'intero ciclo di validazione.

Campi gestiti applicativamente:

Campo	Descrizione
ID	Identificativo richiesta
DATA_ORA	Data e ora avvio richiesta
SERVIZIO_CHIAMANTE	Nome servlet/servizio
ESITO	Stato elaborazione
CF_PAZIENTE	Codice fiscale paziente
CF_OPERATORE	Codice fiscale operatore
PROGRAMMA	Tipo programma
GATEWAY_STATUS	Status restituito dal Gateway
GATEWAY_TITLE	Title errore Gateway
GATEWAY_DETAIL	Dettaglio errore Gateway
GATEWAY_TRACE_ID	Trace ID restituito
GATEWAY_WORKFLOW_INSTANCE_ID	Workflow instance id
JSON_COMPLETO	Risposta completa, ove gestita
MESSAGGIO	Messaggio applicativo

Stati principali

Stato	Descrizione
IN_PROGRESS	Richiesta avviata
SUCCESS	Validazione completata con successo
ERROR	Errore applicativo o Gateway

8.3 Tabelle di log

LOG_TABLE

Tabella utilizzata per registrare eventi applicativi.

Campi gestiti:

Campo	Descrizione
TIPO_LOG	INFO, ERROR, WARNING
MESSAGGIO	Messaggio di log
FUNZIONE	Funzione/servlet chiamante
TIMESTAMP	Data evento
PARAMS	Parametri principali
SERVIZIO	Servizio applicativo
DATI	Dati aggiuntivi

8.4 Accessi richiesti

L'utente Oracle configurato nel DataSource deve disporre almeno dei seguenti permessi:

- connessione al database;
- INSERT su LOG_TABLE;
- INSERT su MONITORAGGIO_GATEWAY;
- UPDATE su MONITORAGGIO_GATEWAY;
- eventuale SELECT su sequence/identity se richiesta;
- eventuali permessi aggiuntivi previsti dal modello dati.

9. Sicurezza

9.1 Sicurezza trasporto

La comunicazione verso il Gateway FSE avviene tramite protocollo HTTPS/TLS.

Il sistema configura un `SSLContext` applicativo basato su certificato client.

Misure previste:

- utilizzo di keystore PKCS#12;
 - protezione password keystore;
 - invio richieste su canale cifrato;
 - autenticazione client verso servizio esterno.
-

9.2 Gestione certificati

I certificati sono elementi critici del sistema.

Devono essere protetti tramite:

- permessi filesystem restrittivi;
 - accesso consentito solo all'utente del servizio Tomcat;
 - backup controllato;
 - rotazione secondo scadenza certificato.
-

9.3 Gestione JWT

I JWT vengono generati dinamicamente per ogni richiesta.

Caratteristiche:

- algoritmo RS256;
- header con algoritmo, tipo token e certificato pubblico;
- claim generati in base al tipo programma;
- issuer differenziato;
- scadenza configurata;
- firma con chiave privata estratta dal P12.

I token non devono essere scritti nei log applicativi, salvo eventuali identificativi tecnici non sensibili.

9.4 Cifratura password database

La password del database è cifrata nel context Tomcat.

Algoritmo

AES/GCM/NoPadding

Chiave

GATEWAYFSE_DB_SECRET_KEY

Formato consigliato nel context

```
password="ENC (PASSWORD_CIFRATA) "
```

9.5 Permessi filesystem

L'utente del servizio Tomcat deve avere permessi adeguati su:

- directory applicazione;
- directory log;
- directory certificati;
- directory temporanea Java;
- eventuali directory di configurazione.

Permessi raccomandati:

Directory	Permessi
Applicazione WAR	Lettura/esecuzione
Log	Lettura/scrittura
Certificati	Lettura
Temp	Lettura/scrittura/cancellazione
Configurazione	Lettura

10. Performance e limiti

10.1 Dimensione file

Il sistema riceve file tramite multipart e li salva temporaneamente sul filesystem.

Limiti da valutare/configurare:

- dimensione massima upload Tomcat;
 - dimensione massima richiesta HTTP;
 - spazio disponibile nella directory temporanea;
 - timeout chiamata esterna;
 - memoria disponibile JVM.
-

10.2 Timeout chiamate esterne

Le chiamate coinvolte sono:

- richiesta access token;
- invocazione Gateway validazione.

È opportuno configurare timeout coerenti con:

- dimensione documenti;
 - latenza rete;
 - prestazioni Gateway;
 - requisiti operativi.
-

10.3 Gestione concorrenza

Tomcat può gestire più richieste contemporanee.

Aspetti da considerare:

- pool connessioni Oracle;
- thread Tomcat;
- accesso concorrente alla directory temporanea;
- uso di variabili statiche;
- tempo medio di risposta del Gateway.

Il DataSource è configurabile con parametri come `maxActive` e `initialSize`, che regolano il pool connessioni database.

10.4 Limiti operativi

Ambito	Limite / Considerazione
Upload file	Dipende dalla configurazione multipart/Tomcat
Connessioni DB	Dipende da <code>maxActive</code>
Certificati	Devono essere validi e non scaduti
Token	Dipende dal servizio di autenticazione
Gateway	Dipende dalla disponibilità del servizio esterno
Temp filesystem	Deve avere spazio sufficiente

11. Deployment

11.1 Build applicazione

Il progetto è gestito tramite Maven.

Esempio build:

```
mvn clean package
```

Output atteso:

```
target/GatewayFSE.war
```

11.2 Deploy su Tomcat 9

Procedura

1. Arrestare il servizio Tomcat.
2. Copiare il file WAR in:

```
<TOMCAT_HOME>\webapps\
```

3. Configurare il context applicativo.
 4. Verificare il file `config.properties`.
 5. Verificare la presenza dei certificati.
 6. Configurare la variabile d'ambiente `GATEWAYFSE_DB_SECRET_KEY`.
 7. Avviare Tomcat.
 8. Verificare i log di avvio.
 9. Eseguire una chiamata di test.
-

11.3 Configurazione ambiente

Elementi da configurare per ogni ambiente:

Elemento	Configurazione
Context path	/GatewayFSE
DataSource Oracle	context Tomcat
Password DB cifrata	context Tomcat
Chiave AES	variabile d'ambiente
Endpoint token	config.properties
Endpoint validazione	config.properties
Certificati	filesystem applicativo
Log	directory configurata
Java	Java 21

Tomcat	Tomcat 9
--------	----------

12. Requisiti di sistema

12.1 Requisiti hardware

Risorsa	Minimo	Consigliato
CPU	2 core	4 core o superiore
RAM	8 GB	16 GB
Disco	Spazio sufficiente per log/temp	SSD consigliato
Rete	Accesso DB e Gateway	Connessione stabile e monitorata

12.2 Requisiti software

Software	Versione
Sistema operativo	Windows Server 2022
Java	Java 21
Application server	Apache Tomcat 9
Build tool	Maven
Database	Oracle
Driver JDBC	Oracle JDBC compatibile
Librerie HTTP	OkHttp
Librerie JSON	Jackson
Librerie JWT	JJWT
Provider crypto	BouncyCastle

12.3 Requisiti rete

Il server applicativo deve poter raggiungere:

Destinazione	Protocollo	Utilizzo
Database Oracle	TCP	Logging e monitoraggio
Token service	HTTPS	Ottenimento access token
Gateway Regionale FSE	HTTPS/TLS	Validazione documento

13. Ambiente di esercizio

13.1 Componenti dell'ambiente

Server applicativo (PVESBASL5)

Elemento	Valore
Sistema operativo	Windows Server 2022
Application server	Apache Tomcat 9
Runtime	Java 21
Applicazione	GatewayFSE
Modalità deploy	WAR

Server database (TOPDBF)

Elemento	Valore
Tecnologia	Oracle
Accesso	JNDI DataSource Tomcat
Utilizzo	Log e monitoraggio

Servizi esterni (Liguria Digitale)

Sistema	Funzione
Token service	Rilascio access token
Gateway Regionale FSE	Validazione documento
Infrastruttura certificati	Autenticazione TLS e firma JWT

Filesystem applicativo (PVESBASL5)

Area	Funzione
Configurazione	File <code>config.properties</code>
Certificati	File P12/PKM
Log	Log applicativi
Temp	File temporanei ricevuti in upload

13.2 Modalità di esecuzione

Il sistema opera in modalità server-side, esposto tramite Tomcat.

L'elaborazione viene attivata da chiamata HTTP verso la servlet applicativa.

Ogni richiesta è indipendente e segue il ciclo:

```
Ricezione input
→ Generazione token
→ Configurazione TLS
→ Invocazione Gateway
```

→ Aggiornamento monitoraggio
→ Risposta JSON

13.3 Considerazioni operative

Il sistema è progettato per:

- funzionare senza interfaccia utente diretta;
- ricevere richieste applicative interne;
- centralizzare la logica di validazione verso Gateway FSE;
- tracciare ogni richiesta su database;
- mantenere configurazioni esterne al codice;
- proteggere password database tramite cifratura;
- selezionare certificati in base al tipo programma;
- restituire risposte JSON coerenti.

Eventuali anomalie sono gestite tramite:

- log applicativo;
 - log database;
 - risposta JSON di errore;
 - aggiornamento del monitoraggio;
 - conservazione dei trace id restituiti dal Gateway.
-

13.4 Sintesi componenti

Componente	Tecnologia	Posizione	Ruolo
GatewayFSE	Java 21 / WAR	Tomcat 9	Validazione documenti tramite Gateway
DocumentsValidation	Servlet Java	Applicazione web	Endpoint principale
Repository	Java / JDBC / JNDI	Applicazione web	Logging e monitoraggio DB
Oracle DB	Oracle	Server database	Persistenza log e monitoraggio
Token Service	HTTPS	Esterno	Rilascio access token
Gateway Regionale FSE	HTTPS/TLS	Esterno	Validazione documento
Certificati P12/PEM	PKCS#12 / PEM	Filesystem server	TLS e firma JWT
PasswordCrypto	Java Crypto API	Applicazione web	Cifratura/decifratura password DB
EncryptedDataSourceFactory	Tomcat JDBC Pool	Context Tomcat	Decifratura password DataSource